

Feature Fusion Using Locally Linear Embedding for Classification

Bing-Yu Sun, Xiao-Ming Zhang, Jiuyong Li, and Xue-Min Mao

Abstract—In most complex classification problems, many types of features have been captured or extracted. Feature fusion is used to combine features for better classification and to reduce data dimensionality. Kernel-based feature fusion methods are very effective for classification, but they do not reduce data dimensionality. In this brief, we propose an effective feature fusion method using locally linear embedding (LLE). The proposed method overcomes the limitations of LLE, which could not handle different types of features and is inefficient for classification. We propose an efficient algorithm to solve the optimization problem in obtaining weights of different features, and design an efficient method for LLE-based classification. In comparison to other kernel-based feature fusion methods, the proposed method fuses features to a significantly lower dimensional feature space with the same discriminant power. We have conducted experiments to demonstrate the effectiveness of the proposed feature fusion method.

Index Terms—Dimension reduction, feature fusion, locally linear embedding, supervised learning.

I. INTRODUCTION

With the increasing number of data capture devices and feature extraction methods, many different types of features have been obtained. The fusion of different feature vectors is in great demand in many applications, such as computer vision, remote sensing, robotics, and bioinformatics. Feature fusion becomes a necessary data processing step and has been shown promising in classification [1]–[3].

The primary goal of feature fusion is to make use of a wide range of complementary information on all possible characteristics of objects for classification. However, various feature vectors may contain redundant and irrelevant information. It is desirable to remove the redundant and irrelevant information in feature fusion process. Feature reduction is necessary for many applications. For example, features in many biomedical data sets outnumber samples. The reduction of features is desirable for deterministic solutions. In many distributed systems, such as video surveillance systems, data sharing is essential. Low-dimensional data reduces communication load and improves communication efficiency. Therefore, feature reduction is also a major goal of feature fusion.

A straightforward feature fusion approach is to concatenate various feature vectors to a single feature vector, which is of high dimension. Feature concatenation has achieved certain successes [4], [5], but it suffers from the curse of dimensionality and uses certain information repeatedly. Alternatively to the feature concatenation, we can build multiple classifiers on different types of features and combine these clas-

sifiers for classification [6]–[9]. The key of classifier combination is a good strategy to combine outputs of different classifiers. Classifier combination can be an effective approach for classification, but parameter tuning for multiple classifiers and adjusting integration module can be a daunting task. Further, it is time consuming to build multiple classifiers.

Recently, kernel-based methods have been used to solve feature fusion problems. These methods learn a kernel matrix from different features via different algorithms, such as semidefinite programming method, Bayesian hierarchical models, and so on [10]–[13]. With the obtained kernel matrix, different types of features from various sources can be fused. Although the kernel-based methods have improved the nonlinear discriminability, they do not reduce the dimensionality of data space. Further, the choice of right kernels for right feature vectors is challenging.

This brief proposes a novel feature fusion method using locally linear embedding (LLE). Recently, a conceptually simple yet powerful method for feature extraction, LLE, has been proposed by [14]. The central idea of LLE is to solve globally nonlinear problems using the locally linear fitting. LLE is based on the assumption that data lying on a nonlinear manifold can be viewed as linear in local areas. However, existing LLE methods take only one type of features. When several types of features are presented, an LLE algorithm is applied to different types, respectively, and different subspaces are then combined by using a decision combination strategy. The relationships across different feature spaces may not be used and redundant information within different feature vectors may be repeatedly used for classification. In view of the limitations of LLE, we propose a new feature fusion method using LLE. Unlike those traditional LLE algorithms, the proposed method integrates several types of features into a single embedding. That is, we are able to find a single integrated low-dimensional subspace where redundant and irrelevant information are eliminated. In comparison to kernel-based methods, the proposed method fuses features to a significantly lower dimensional feature space with the same discriminant power.

The rest of this brief is organized as follows. Section II introduces the basics of LLE. Section III elaborates on our proposed method for feature fusion. In Section IV, we present experiments and results showing the performance of our proposed methods. Section V concludes the brief.

II. LOCALLY LINEAR EMBEDDING

The LLE maps a data set $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \in R^{d \times n}$ globally to a data set $Y = \{\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n\} \in R^{m \times n}$. The data set (X, Y) is used to model the subsequently relationship. The basic principle of LLE is to minimize the reconstruction error of the set of all local neighborhoods in the data set. This takes two stages: 1) locally fitting hyperplanes around each samples $\mathbf{x}$ based on its K nearest neighbors and calculating reconstruction weights and 2) finding low-dimensional coordinates $\mathbf{y}_i$ for each $\mathbf{x}_i$ by minimizing a mapping function based on these weights.

In stage 1, the cost function to be minimized is

$$\varepsilon_W = \sum_{i=1}^n \left\| \mathbf{x}_i - \sum_{j=1}^K w_{i,j} \mathbf{x}_{N(j)} \right\|^2. \quad (1)$$

This stage is to make $\mathbf{x}_i$ be linearly reconstructed in terms of its neighbors $\mathbf{x}_{N(1)}, \mathbf{x}_{N(2)}, \dots, \mathbf{x}_{N(K)}$. If $\mathbf{x}_i$ and $\mathbf{x}_j$ are not in the same neighborhood, $w_{i,j} = 0$. Weights $w_{i,j}$ are computed according to the least

Manuscript received September 20, 2008; revised August 21, 2009; accepted October 29, 2009. First published December 04, 2009; current version published January 04, 2010. This work was supported by the National Key Technology R&D Program of China under Grant 2008BAK49B05, by the National Science Foundation of China under Grant 70801025, and by the Anhui Provincial Natural Science Foundation under Grant 070416251.

B.-Y. Sun, X.-M. Zhang, and X.-M. Mao are with the Institute of Intelligence Machines, Chinese Academy of Sciences, Hefei, Anhui 230031, China (e-mail: bysun@ustc.edu).

J. Li is with the School of Computer and information Science, University of South Australia, Adelaide, S.A. 5001, Australia.

Color versions of one or more of the figures in this brief are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TNN.2009.2036363

square principle and they sum up to 1. Weights $w_{i,j}$ are stored in an $n \times n$ sparse matrix W .

In stage 2, the weight matrix W is fixed and new m -dimensional vectors $\mathbf{y}_i$ are sought by minimizing the criterion

$$\varepsilon_Y = \sum_{i=1}^n \left| \mathbf{y}_i - \sum_{j=1}^K w_{i,j} \mathbf{y}_{N(j)} \right|^2. \quad (2)$$

Rewriting (2) then gives

$$\varepsilon_Y = \sum_{i=1}^n \sum_{j=1}^n M_{i,j} \mathbf{y}_i^T \mathbf{y}_j = \text{tr}(YMY^T) \quad (3)$$

where M is an $n \times n$ matrix found as $M = (I - W)^T(I - W)$ and Y constrains the $\mathbf{y}_i$ as its columns. Stage 2 is reduced to an optimization problem as follows:

$$\begin{aligned} \min \varepsilon(Y) &= \text{tr}(YMY^T) \\ \text{s.t.} \quad &\frac{1}{n}YY^T = I. \end{aligned} \quad (4)$$

This is an eigenvalue problem. All eigenvectors of M are solutions, but the eigenvectors corresponding to the smallest eigenvalues minimize the objective function of (4).

III. PROPOSED FUSION METHODS

In this section, we present our proposed methodology for feature fusion in two different situations, namely, simple average and weighted average. We also give an efficient strategy for solving classification problems.

A. Simple Fusion Using LLE

Now suppose we have training samples $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$ and each sample $\mathbf{x}_i$ can be represented by L different features $\{\mathbf{x}_i^l, l = 1, 2, \dots, L\}$. Then, for a sample, we can calculate its local neighbor structure in L observation spaces

$$\mathbf{x}_i^l = \sum_{j=1}^K w_{i,j}^l \mathbf{x}_{N(j)}^l + e_i^l, \quad l = 1, \dots, L \quad (5)$$

where e_i^l is a reconstruction error. Accordingly, L weight matrices $W_1, \dots, W_L$ can be obtained using (1). These K weight matrices encode different structure information and each structure has its strengths and weaknesses. To utilize all the neighbor information, we combine L weight matrices

$$\bar{W} = \frac{\sum_{l=1}^L W^l}{L}. \quad (6)$$

Then, we find new m -dimensional vectors $\mathbf{y}_i$ by solving the following optimization problem:

$$\begin{aligned} \min \varepsilon(Y) &= \text{tr}(Y(I - \bar{W})^T(I - \bar{W})Y^T) \\ \text{s.t.} \quad &\frac{1}{N}YY^T = I. \end{aligned} \quad (7)$$

This is the same optimization problem as (4) and its global value can be obtained through solving an eigenvalue problem.

B. Weighted Feature Fusion Using LLE

The previous method considers that all features have the same importance during the fusion. In practice, various features have different importance. A key issue in feature fusion is to derive a weighted scheme to reflect relative importance of different features. This can be achieved by a weighted method, i.e., assigning larger weights to more important features and smaller weights to less important ones. The weighted fusion method assigns weights ω_l satisfying the following constraints to the l th features:

$$0 \leq \omega_l \leq 1 \quad (8)$$

$$\sum_{l=1}^L \omega_l = 1. \quad (9)$$

We obtain a weighted matrix as follows:

$$\bar{W} = \sum_{l=1}^L \omega_l W^l. \quad (10)$$

New m -dimensional vectors $\mathbf{y}_i$ can be sought by solving the following optimization problem:

$$\begin{aligned} \min \varepsilon(Y, \omega) &= \text{tr} \left\{ Y \left(I - \sum_{l=1}^L \omega_l W^l \right)^T \left(I - \sum_{l=1}^L \omega_l W^l \right) Y^T \right\} \\ \text{s.t.} \quad &\frac{1}{n}YY^T = I, \quad 0 \leq \omega_l \leq 1, \quad \sum_{l=1}^L \omega_l = 1. \end{aligned} \quad (11)$$

Unlike (4) or (7), which is an eigenvalue problem and easily obtains the global optimal solution, objective in (11) is a complicated nonconvex optimization problem. So it is difficult to obtain the global optimal solutions directly.

If we fix the value of ω , we will get an optimization problem as follows:

$$\begin{aligned} \min \varepsilon(Y) &= \text{tr} \left\{ Y \left(I - \sum_{l=1}^L \omega_l W^{lT} \right) \left(I - \sum_{l=1}^L \omega_l W^l \right) Y^T \right\} \\ \text{s.t.} \quad &\frac{1}{n}YY^T = I. \end{aligned} \quad (12)$$

If we fix the value of Y , we obtain another optimization problem as follows:

$$\begin{aligned} \min \varepsilon(\omega) &= \text{tr} \left\{ Y \left(I - \sum_{l=1}^L \omega_l W^{lT} \right) \left(I - \sum_{l=1}^L \omega_l W^l \right) Y^T \right\} \\ \text{s.t.} \quad &0 \leq \omega_l \leq 1 \\ &\sum_{l=1}^L \omega_l = 1. \end{aligned} \quad (13)$$

Problem in (12) is an eigenvalue one and problem in (13) is a quadric programming one. Both can be easily resolved optimally. Based on this decomposition, an alternating optimization technique [19] can be adopted to solve this optimization problem efficiently. The procedure of solving (11) is outlined in Algorithm 1.

Algorithm 1:

- 1: Input:
 - 2: Given a training data set $\{X\}$, the maximal number of iteration: I , the tolerance error: ε_1 , and ε_2
 - 3: Initialize the value of ω and Y . (A simple approach is to set $\omega_l = 1/L, l = 1, \dots, L$ and $Y = 0$)
 - 4: For $i = 1$ to I , do:
 - 5: Get Y^i through (12) while fixing the value of ω^i
 - 6: Get ω^i through (13) while fixing the value of Y^i
 - 7: If $|\omega^i - \omega^{i-1}| > \varepsilon_1$ or $|Y^i - Y^{i-1}| > \varepsilon_2$, $i = i + 1$, goto 4; else, goto 7
 - 8: End for
 - 9: Outputs: ω and Y .
-

Although it is not guaranteed that the global solution of (11) will be reached, local optimal solution is usually satisfactory. The solution is not sensitive to initial points. In our experiments, ten different initial points are used and they cause little deviation to results.

C. LLE for Classification

Previous studies have shown that LLE is effective for visualization [14]. When it is used for classification, it has the following two major limitations.

- 1) In the supervised learning process the label information of samples is available. However, traditional LLE algorithms cannot use this information since LLE is an unsupervised dimension reduction method. To resolve this problem, several methods have been proposed [15], [16]. The intuition of these methods is to create disjoint embedding for individual classes. This is normally achieved by artificially increasing the distances between samples belonging to different classes, and by keeping distances unchanged between samples within the same class. Thus, the label information of training samples is utilized. Although this approach makes LLE work for classification, it has two weaknesses. First, this approach divides all the sample points into disconnected parts instead of an entire graph in the original LLE. A new problem on how to apply the LLE to disconnected components emerges. Second, the approach is not robust to the noise. Therefore, how to utilize label information efficiently is still open for supervised LLE algorithms.
- 2) An LLE algorithm does not directly compute the mapping of test samples because the weighted matrix of LLE is constructed on the training data. A straightforward solution is to map a new sample $\mathbf{x}$ by finding its K nearest neighbors and mapping it using calculated construction weights on these neighbors. A major disadvantage of this method, however, is that it requires access to the entire training set, which demands a large storage. Recently, several proposed methods plugged a linear transformation to the original LLE [17], [18] for solving this out-of-sample problem. The embedding results of the coming points can be successfully attained with the linear transformation. Using linear transformation overcomes the out-of-sample problem with the low computational cost.

To overcome the above two limitations, we can apply a linear transformation to the original LLE as follows:

$$Y = \begin{pmatrix} V_1 \\ V_2 \\ \vdots \\ V_L \end{pmatrix}^T \cdot \begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_L \end{pmatrix} \\ = V^T X, V_l \in R^{d_l \times m} \\ X_l = \{\mathbf{x}_1^l, \mathbf{x}_2^l, \dots, \mathbf{x}_n^l\} \in R^{d_l \times n} \quad (14)$$

where d_l is the dimension of the l th feature. This transformation V maps each $\mathbf{x}_i = [(\mathbf{x}_i^1)^T, \dots, (\mathbf{x}_i^L)^T]^T$ to a lower m -dimensional space. The V is optimized based on the following criteria.

- The V minimizes the reconstructed error so that the information of the original data can be reserved.
- The V maximizes the margin between different classes so that the label information of the data can be used and the classification performance can be improved.

To find such a V , we transform (11) to the following optimization problem:

$$\min \varepsilon(V, \omega) = \text{tr} \left\{ V^T X \left(I - \sum_{l=1}^L \omega_l W^L \right)^T \left(I - \sum_{l=1}^L \omega_l W^l \right) X^T V \right\} \\ - C \cdot \text{tr}(V^T S_b V) \quad (15)$$

$$\text{s.t.} \quad \frac{1}{n} V^T X X^T V = I, \quad 0 \leq \omega_l \leq 1, \quad \sum_{l=1}^L \omega_l = 1$$

where S_b is a between-class scatter matrix, C is a penalty coefficient, and tr is the trace of a matrix. After solving (15), we can obtain a low-dimensional embedding $Y = V^T X$.

Problem in (15) is also a complicated nonconvex optimization problem. To obtain a satisfactory solution, we adopt alternating optimization technique [19] again. The outline of the algorithm is as follows.

Algorithm 2:

- 1: Input:
 - 2: Given a training data set $\{X\}$, the maximal number of iteration: I , the tolerance error: ε_1 , and ε_2
 - 3: Initialize the value of ω and Y . (A simple approach is to set $\omega_l = 1/L, l = 1, \dots, L$ and $Y = 0$)
 - 4: For $i = 1$ to I , do
 - 5: Get V^i through (16) while fixing the value of ω^i
 - 6: Get ω^i through (17) while fixing the value of V
- $$\min \varepsilon(Y) = \text{tr} \left\{ V^T X \left(I - \sum_{l=1}^L \omega_l W^{lT} \right) \left(I - \sum_{l=1}^L \omega_l W^l \right) X^T V \right\} \\ - C \cdot \text{tr}(V^T S_b V) \quad (16)$$
- $$\text{s.t.} \quad \frac{1}{n} V^T X X^T V = I$$
- $$\min \varepsilon(\omega) = \text{tr} \left\{ V^T X \left(I - \sum_{l=1}^L \omega_l W^{lT} \right) \left(I - \sum_{l=1}^L \omega_l W^l \right) X^T V \right\} \quad (17)$$

TABLE I
NUMBER OF THE WRONGLY CLASSIFIED SAMPLES FOR DIFFERENT FEATURES FOR UCI HANDWRITTEN DIGIT DATA SET

Feature	Original Feature						Fused Feature		
	FA	FO	KA	MO	PI	ZE	ALL	SF	WF
Dimension	216	76	64	6	240	47	649	7	8
Error	114.2	391.5	195.2	518.6	96.5	380.4	85.1 ± 3.4	79.4 ± 2.2	62.5 ± 2.7

$$\text{s.t. } 0 \leq \omega_l \leq 1, \quad \sum_{l=1}^L \omega_l = 1$$

7: If $|\omega^i - \omega^{i-1}| > \varepsilon_1$ or $|V^i - V^{i-1}| > \varepsilon_2$, $i = i + 1$, goto 4;
 else, goto 7
 8: End for
 9: Outputs: ω and V .

D. Estimation of Intrinsic Dimension

The dimension of embedding is a key parameter for LLE methods. So far many approaches have been proposed to estimate the intrinsic dimension of the embedding [20]–[22]. Recently, Farahmand *et al.* proposed an efficient algorithm for estimating the unknown dimension of a manifold from samples using the following [22]:

$$m(\mathbf{x}) = \frac{\ln 2}{\ln(r^p(\mathbf{x})/r^{\lceil p/2 \rceil}(\mathbf{x}))} \quad (18)$$

where $m(\mathbf{x})$ is the dimension estimated from p neighbors of sample $\mathbf{x}$, $r^p(\mathbf{x})$ is the distance to the p th nearest neighbor of $\mathbf{x}$, and $\lceil p/2 \rceil$ denotes the rounded value of $p/2$. The final estimated dimension of the data can be computed as follows:

$$m_{\text{avg}} = \left\lceil \frac{\sum_{i=1}^n (m(\mathbf{x}_i) \wedge d)}{n} \right\rceil. \quad (19)$$

Here, $a \wedge b = \min(a, b)$ and d is the original dimension of $\mathbf{x}_i$.

When several features are available, to determine the dimension of latent feature, we can first compute $m(\mathbf{x}_i^l)$, $i = 1, \dots, n$; $l = 1, \dots, L$ using (18) and the final dimension can be obtained with the following:

$$m_{\text{avg}} = \left\lceil \frac{\sum_{l=1}^L \omega_l \sum_{i=1}^n (m(\mathbf{x}_i^l) \wedge d_l)}{n} \right\rceil. \quad (20)$$

A problem with (20) is that the value of ω_l is unknown in advance. In practice, we can compute ω_l through Algorithm 2, then calculate m_{avg} using (20), and select the m_{avg} bottom generalized eigenvalues and the corresponding eigenvectors matrix obtained through (16) to get embedding V .

IV. EXPERIMENTAL RESULTS

To evaluate the performance of our proposed methods for feature fusion, we performed a number of experiments using two data sets, namely, University of California at Irvine (UCI) handwritten digit data set [23] and Brodatz's texture image data set [24]. Both data sets contain multiple feature vectors of different types. For each data set, we concatenated all feature vectors to form a long feature vector (ALL) for the comparison. Parameters of LLE were determined by using a cross-validation method. The value of p of (18) is set to $\lceil 2 \ln n \rceil$ when we es-

TABLE II
NUMBER OF THE WRONGLY CLASSIFIED SAMPLES FOR DIFFERENT METHODS FOR UCI HANDWRITTEN DIGIT DATA SET

Methods	#Features	Error
SDP	649	60.8 ± 2.4
Bayesian	649	61.1 ± 1.4
SF	7	62.3 ± 1.8
WF	8	60.5 ± 2.1

timated the intrinsic dimension of data as in [22]. We adopted nearest neighbor (NN) method [27] to evaluate the performances of three feature fusion methods, namely, ALL method, weighted fusion method, and simple fusion method. To compare the classification performance, two kernel-based feature fusion methods, namely, SDP method of [10] and Bayesian method of [11] were used and their performances are compared with a support vector machine (SVM) classifier trained on the feature obtained with the proposed two feature fusion methods. To solve the semidefinite programs of [10] and the posterior distribution estimation of [11], the general purpose optimization software SVX [25] and the variational Bayesare algorithm provided in [26] with their default setting are used. We adopted a one-versus-all method to realize the multiclass classification [28]. The following Gaussian kernel was used in our experiments:

$$k(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{y}\|^2}{\sigma^2}\right). \quad (21)$$

Tenfold cross validation was used to determine parameters. We averaged the results over ten trials to fairly compare our results with those of other algorithms. A significant difference is declared if the p value from the t -test was less than 0.05.

A. UCI Handwritten Digit Data Set

The data set consists of features of handwritten numerals (0–9) extracted from a collection of Dutch utility maps. Two hundred patterns per digit class (total of 2000 patterns) have been digitized as binary images. These digits are represented in terms of the following six different feature types: 1) profile correlations (FA); 2) Fourier coefficients of the character shapes (FO); 3) Karhunen–Loeve (KA) coefficients; 4) morphological features (MO); 5) pixel averages in 23 windows (PI); and 6) Zernike moments (ZE). From the 200 patterns for each digit class, 50 samples were selected randomly for obtaining the low-dimensional space and the remaining data are mapped to this space. A single long feature, formed by concatenating these six features, is also used for the comparison.

The first set of experiments is to evaluate the performance of our proposed feature fusion methods. Nine different feature vectors, including

TABLE III
NUMBER OF THE WRONGLY CLASSIFIED SAMPLES FOR DIFFERENT FEATURES FOR TEXTURE IMAGE DATA SET

Feature	Original Feature					Fused Feature		
	ACF	EF	WTA	DCT	GB	ALL	SF	WF
Dimension	20	100	7	8	16	151	5	5
Error	174.9	80.2	228.3	487.1	256.8	67.1 ± 11.2	28.4 ± 4.1	16.1 ± 3.8

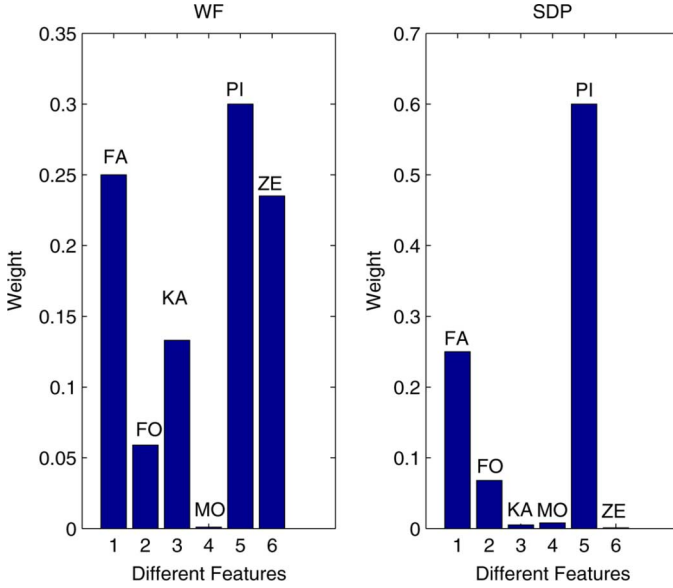


Fig. 1. Weights of different features for UCI handwritten digit data set.

the above six individual feature vectors, the single long feature vector formed by concatenating all individual feature vectors, and feature vectors formed by simple fusion and weighted fusion were used. Their corresponding classification errors when using 1-NN were recorded. Table I shows the number of the wrongly classified samples when different feature vectors are used. From this table it can be seen that the performances of different feature vectors vary greatly. However, by using our proposed fusion approach, the performance of 1-NN classification is the best and the dimension of the fused feature is significantly lower than that of original ones.

Table II shows the classification results of four different feature fusion methods, namely, SDP, Bayesian, SF, and WF. It can be seen that the performances of these four methods are similar. Significance test shows that there is no statistically significant difference between the performances of the proposed weighted fusion method and two kernel-based fusion methods (SDP: $p = 0.64$, Bayesian: $p = 0.12$). Consider that the dimension of the feature space is only 7 or 8 after the proposed feature fusion process in comparison to hundreds of others. Such competitive classification accuracy is very impressive.

The weights of each individual feature obtained through our proposed method and the SDP method are shown in Fig. 1, from which the differences between these two approaches can be seen. Two features with the largest weight obtained through WF method are the same as those obtained through SDP method (FA feature and PI feature). Interestingly, these two feature also obtain the best classification performance when single feature is used. Assigning great weights to strong features is reasonable.

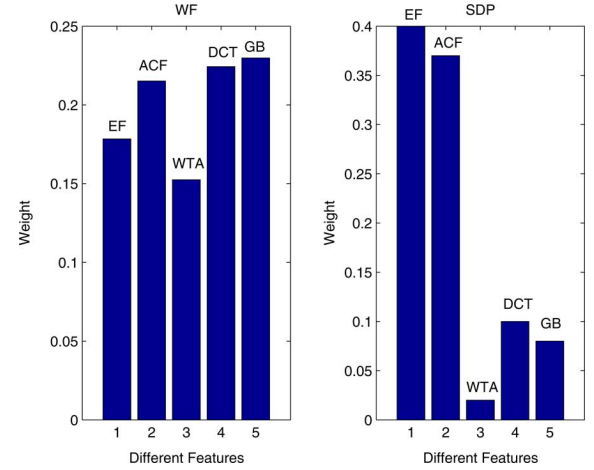


Fig. 2. Weights of different features for texture image data set.

B. Texture Image Data Set

Nine texture images from Brodatz's texture album (D1, D3, D6, D11, D16, D17, D20, D21, D24) are used for texture classification experiments. Each image is of size 512×512 pixels with 256 gray values. From each original image, 200 subimages of size 32×32 are extracted randomly; 50 of them are used for training and the remaining 150 images for testing. Five most commonly used feature types, including autocorrelation (ACF), edge frequency (EF), wavelet transform (WT), discrete cosine transform (DCT), and Gabor transform (GB), were used to represent the texture images [19]. The single long feature vector formed by concatenating the above five feature vectors was also used to represent the images for the classification experiments. Table III shows the number of the wrongly classified samples when different features are used. It can be seen that using multiple features can improve the performance of classification in comparison with the best performance achievable by any single feature. Moreover, the table also shows that our proposed combination approaches outperform other methods and the proposed weighted fusion outperforms the proposed simple fusion.

Table IV shows the classification results when different feature fusion methods are used. The proposed weighted fusion method performs better than SDP ($p = 0.003$), and performs similarly to Bayesian ($p = 0.2$). The fused feature space of the proposed weighted fusion method is significantly lower than those of kernel-based methods. This demonstrates that the proposed method is able to achieve the same classification performance with a lower feature space. Fig. 2 shows the weights of each individual feature obtained through WF method and the SDP method. Although the distribution of the weights of used features changes significantly when different fusion methods are used, the achieved classification performances are similar. This can be explained by the following fact. When several features are used, they are normally not independent from one another strictly. So if two related features

TABLE IV
NUMBER OF THE WRONGLY CLASSIFIED SAMPLES FOR DIFFERENT
METHODS FOR TEXTURE IMAGE DATA SET

Methods	#Feature	Error
SDP	151	14.4 ± 2.9
Bayesian	151	12.9 ± 2.3
SF	5	14.9 ± 1.8
WF	5	13.5 ± 3.1

have similar classification performances, such as the WTA feature and the DCT feature of this problem, it is very difficult to assign them reasonable and fixed weights.

V. CONCLUSION

This brief studies the use of LLE for feature fusion. We have presented two LLE-based feature fusion methods: simple and weighted feature fusions. The proposed methods transform all available features of different types to a single low-dimensional feature space. To obtain the optimal weights of different features, an efficient method has been developed for solving a complicated nonconvex optimization problem. We also presented an efficient method for solving classification problems by LLE. The performance of the proposed methods has been evaluated by using two data sets, namely, UCI handwritten digit data set and Brodatz's texture image data set. The experimental results show that the proposed feature fusion methods outperform the feature concatenation method which concatenates all features. The proposed weighted feature fusion method achieves the same classification accuracy as two typical kernel-based feature fusion methods but with a significantly lower fused feature space than those of the kernel-based feature fusion methods. Future work will address some issues related to the use of the proposed feature fusion methods in practice, such as how to evaluate the importance of features.

REFERENCES

- [1] L. I. Kuncheva, *Combining Pattern Classifiers, Methods and Algorithms*. New York: Wiley, 2004.
- [2] S. Osowski, L. T. Hoai, and T. Markiewicz, "Support vector machine-based expert system for reliable heartbeat recognition," *IEEE Trans. Biomed. Eng.*, vol. 51, no. 4, pp. 582–589, Apr. 2004.
- [3] A. K. Jerebkoa, J. D. Malleyb, M. Franaszeka, and R. M. Summers, "Computer-aided polyp detection in CT colonography using an ensemble of support vector machines," in *Int. Proc. Congr. Ser.*, 2003, pp. 1019–1024, 1256.
- [4] Z. Su, H. Zhang, S. Li, and S. Ma, "Relevance feedback in content-based image retrieval: Bayesian framework, feature subspaces, and progressive learning," *IEEE Trans. Image Process.*, vol. 12, no. 8, pp. 924–937, Aug. 2003.
- [5] R. D. Zilca and Y. Bistriz, "Feature concatenation for speaker identification," in *Proc. 10th Eur. Signal Process. Conf.*, Tampere, Finland, 2000, pp. 453–456.
- [6] C. Y. Suen, C. Nadal, T. A. Mai, R. Legault, and L. Lam, "Recognition of totally unconstrained handwritten numerals based on the concept multiple experts," in *Proc. Int. Workshop Frontiers Handwriting Recognit.*, C. Y. Suen, Ed., Montreal, Canada, Apr. 2–3, 1990, pp. 131–143.
- [7] C. Nadal, R. Legault, and C. Y. Suen, "Complementary algorithms for the recognition of totally unconstrained handwritten numerals," in *Proc. 10th Int. Conf. Pattern Recognit.*, Jun. 1990, pp. 434–449.
- [8] J. J. Hull, A. Commike, and T. K. Ho, "Multiple algorithms for handwritten character recognition," in *Proc. Int. Workshop Frontiers Handwriting Recognit.*, Apr. 1990, pp. 117–124.
- [9] T. K. Ho, J. J. Hull, and S. N. Srihari, "Combination of structural classifiers," in *Proc. Workshop Syntactic Structural Pattern Recognit.*, Jun. 1990, pp. 123–137.
- [10] G. R. G. Lanckriet, N. Cristianini, P. Bartlett, L. E. Ghaoui, and M. I. Jordan, "Learning the Kernel matrix with semidefinite programming," *J. Mach. Learn. Res.*, vol. 5, pp. 27–72, 2004.
- [11] M. Girolami and S. Rogers, "Hierarchic Bayesian models for kernel learning," in *Proc. 22nd Int. Conf. Mach. Learn.*, Bonn, Germany, 2005, pp. 241–248.
- [12] D. Girolami and M. A. Girolami, "Combining feature spaces for classification," *Pattern Recognit.*, vol. 42, no. 11, pp. 2671–2683, 2009.
- [13] D. Girolami and M. A. Girolami, "Pattern recognition with a Bayesian kernel combination machine," *Pattern Recognit. Lett.*, vol. 30, no. 1, pp. 46–54, 2009.
- [14] L. K. Saul and S. T. Roweis, "Nonlinear dimensionality reduction by locally linear embedding," *Science*, vol. 290, pp. 2323–2326, 2000.
- [15] D. Ridder and R. P. W. Duin, "Locally linear embedding for classification," *Imag. Sci. Technol. Dept., Faculty Appl. Sci., Delft Univ. Technol., The Pattern Recognition Group, Tech. Rep. Ser. PH-2002-01*, 2002.
- [16] O. Kouropteva, O. Okun, A. Hadid, M. Soriano, S. Marcos, and M. Pietikainen, "Beyond locally linear embedding algorithm," *Mach. Vis. Group, Univ. Oulu, Oulu, Finland, Tech. Rep. MVG-01-2002*, 2002.
- [17] E. Kokipoulou and Y. Saad, "Orthogonal neighborhood preserving projections," in *Proc. 5th IEEE Int. Conf. Data Mining*, 2005, pp. 1–7.
- [18] B. Li and D.-S. Huang, "Locally linear discriminant embedding: An efficient method for face recognition," *Pattern Recognit.*, vol. 41, no. 12, pp. 3813–3821, 2008.
- [19] J. C. Bezdek1 and R. J. Hathaway, "Some notes on alternating optimization," in *Lecture Notes in Artificial Intelligence*, N. R. Pal and M. Sugeno, Eds. Berlin, Germany: Springer-Verlag, 2002, vol. 2275, pp. 288–300.
- [20] E. Levina and P. J. Bickel, "Maximum likelihood estimation of intrinsic dimension," in *Advances in Neural Information Processing Systems (NIPS 17)*. Cambridge, MA: MIT Press, 2005, pp. 777–784.
- [21] M. Hein and J.-Y. Audibert, "Intrinsic dimensionality estimation of submanifolds in Euclidean space," in *Proc. 22nd Int. Conf. Mach. Learn.*, 2005, pp. 289–296.
- [22] A. M. Farahmand, C. Szepesvari, and J.-Y. Audibert, "Manifold-adaptive dimension estimation," in *Proc. 24th Int. Conf. Mach. Learn.*, 2007, pp. 265–272.
- [23] C. Blake, UCI Repository of Machine Learning Databases, Dept. Inf. Comput. Sci., Univ. California Irvine, Irvine, CA, 1998 [Online]. Available: <http://www.ics.uci.edu/mllearn/MLRepository.html>
- [24] P. Brodatz, *Textures: A Photographic Album for Artists and Designers*. New York: Dover, 1966.
- [25] M. Grant and S. Boyd, CVX: Matlab Software for Disciplined Convex Programming (Web Page and Software), Feb. 2009 [Online]. Available: <http://stanford.edu/boyd/cvx>
- [26] S. Rogers, "Hierarchic Bayesian models for kernel learning—Supplementary information," 2009 [Online]. Available: http://www.dcs.gla.ac.uk/~srogers/kernel_comb.html
- [27] G. Shakhnarovich, T. Darrell, and P. Indyk, *Nearest-Neighbor Methods in Learning and Vision*. Cambridge, MA: MIT Press, 2005.
- [28] C.-W. Hsu and C.-J. Lin, "A comparison on methods for multi-class support vector machines," *IEEE Trans. Neural Netw.*, vol. 13, no. 2, pp. 415–425, Mar. 2002.